

Claude Code Build Prompt — mdreader

Paste everything below into Claude Code as the initial prompt. This is a specification, not a conversation. Follow the build phases in order and stop at each gate.

0. Mission

Build **mdreader** — a free, open-source Windows desktop application that opens `.md` files in a clean rendered **Reader** view by default, with a one-keystroke toggle to a **Source** view for editing. Ships as an MSI. Registers itself as a handler for Markdown files and politely offers to become the default on first run.

The target experience: double-click a `.md` file, see a well-typeset document in under a second, press `Ctrl+E`, edit the raw markdown, `Ctrl+S`, back to Reader. Nothing else. No vault, no notebook, no sync, no account.

Positioning: a document *reader* that happens to edit, not an editor that happens to preview. Every design decision resolves in favor of fast, beautiful reading.

1. Technology decisions (these are settled — do not substitute)

Concern	Choice	Rationale
Language / runtime	C# on .NET 10 (LTS)	Verify the current LTS at build time and tell me if it has moved
UI shell	WPF	Mature, minimal, trivial MSI deployment and shell integration; WinUI 3 adds packaging friction for no gain here
Rendering host	WebView2 (Evergreen runtime)	Real HTML/CSS typography — the only way to hit the visual target
Markdown parser	Markdig	CommonMark + GFM, pipeline-configurable, fast, BSD-2

Concern	Choice	Rationale
HTML sanitizer	HtmlSanitizer (ganss)	Mandatory — see §5
Code editor	Monaco Editor , bundled locally	Familiar keybindings, quality editing. If bundle size becomes a problem, CodeMirror 6 is the approved fallback — flag it to me before switching
Syntax highlighting	highlight.js , bundled locally	Shiki is prettier but heavy; leave a seam for it
Diagrams	Mermaid , bundled, <code>securityLevel: 'strict'</code>	
Math	KaTeX , bundled	
Installer	WiX Toolset v7	See the licensing note in §8.1 before you write a line of it
Tests	xUnit + FluentAssertions; Verify for golden-file snapshot tests	
License	MIT	

Everything ships offline. No CDN references anywhere. The app must render a complex document correctly on a machine with no network connection. This is a hard acceptance criterion.

2. Repository layout

```
mdreader/  
  src/  
    MdReader.App/           WPF host, windows, tabs, commands, settings  
    MdReader.Core/          Markdown pipeline, sanitizer, HTML assembly, theming  
    MdReader.Shell/         File association registration, ProgId, shell notify  
    MdReader.Web/           Bundled web assets: reader.html, editor.html, css, js, vendor  
    MdReader.Cli/           Optional console entry point for headless conversion  
  installer/  
    MdReader.Installer/     WiX v7 project  
  tests/  
    MdReader.Core.Tests/  
    MdReader.Shell.Tests/  
    MdReader.Integration.Tests/  
  fixtures/                 .md documents + expected .html snapshots  
  docs/  
  .github/workflows/
```



3. Functional requirements

3.1 Modes

Reader mode (default). Rendered document in a WebView2. Read-only. Selectable, copyable.

Source mode. Monaco editor with markdown syntax highlighting, line numbers, word wrap on, bracket matching, and markdown-aware `Tab` behavior in lists.

Split mode (optional, off by default). Source left, Reader right, with synchronized scrolling.

Toggle: `Ctrl+E` cycles Reader $\rightleftharpoons$ Source. `Ctrl+Shift+E` toggles Split. The View menu and a status-bar button do the same. **Scroll position must be preserved across mode switches** — map source line numbers to rendered block elements by emitting `data-source-line` attributes during rendering, then scroll to the nearest anchor. This is the detail that separates a good implementation from an annoying one; do not skip it.

3.2 Markdown support

CommonMark plus these Markdig extensions, configured explicitly in one place:

pipe tables, grid tables, task lists, strikethrough, subscript/superscript, footnotes, definition lists, auto-identifiers (heading anchors), auto-links, emphasis extras, citations, abbreviations, fenced code with language info, generic attributes, YAML front matter, emoji shortcodes, and custom containers.

Behavior details:

- **YAML front matter** is parsed and rendered as a collapsed metadata card at the top of the document, not dumped as text and not silently discarded.
- **Fenced code blocks** get highlight.js treatment plus a hover "copy" button.
- **Mermaid** blocks (````mermaid`) render as diagrams; on parse failure, fall back to showing the code block with an inline error note — never a blank space.
- **KaTeX** for `$ ... $` and `$$ ... $$`.
- **Task list checkboxes** render as real checkboxes, disabled in Reader mode.
- **Relative image paths** resolve against the document's directory.
- **Tables** get a horizontal scroll container so wide tables never break the layout.
- A generated **table of contents** is available in a collapsible left rail (`Ctrl+Shift+0`), auto-hidden for documents with fewer than 3 headings.

3.3 Visual design

The goal is a calm, readable document, not a code editor with a preview pane. Specifics:

- Content column capped at ~72–78 characters (about 720px at the default size), centered, with generous margins.
- System-UI font stack for prose (Segoe UI Variable on Windows 11), Cascadia Code / Consolas for code.
- Base 16px, 1.65 line-height, clear vertical rhythm; headings with real hierarchy and generous space above, tight below.
- Tables: subtle row separators, no heavy grid lines, header row differentiated by weight not color.
- Blockquotes: left rule, slightly muted text, no background fill.
- Code blocks: soft surface tint, rounded corners, no border.
- Light and dark themes, following the Windows app theme by default, overridable in settings. Both themes need a properly contrast-checked palette (WCAG AA minimum for body text).
- Zoom: `Ctrl + / - / 0`, persisted per app not per file.

Ship the stylesheet as a single, well-commented `reader.css` using CSS custom properties for the whole palette so themes are a variable swap. A user should be able to drop a replacement CSS file into `%APPDATA%\mdreader\themes\` and select it from settings — implement that loader.

3.4 File and window behavior

- **Single instance with tabs.** Opening a second file activates the running instance and adds a tab. Implement with a named mutex plus a named pipe for handoff; handle the race correctly.
- **Live reload.** `FileSystemWatcher` on the open document. If the file changes on disk and the buffer is clean, reload and preserve scroll position. If the buffer is dirty, show a non-modal bar offering Reload / Keep mine. Debounce 250ms — editors and generators write in bursts.
- **Dirty state** in the tab title and window title; confirm on close; `Ctrl+S` saves, preserving the file's original encoding and line endings (detect BOM, CRLF vs LF, and round-trip them faithfully — silently rewriting line endings will corrupt someone's Git diff).
- **Drag and drop** files onto the window.
- **Recent files** list, max 20, in the File menu and on the empty-state screen.
- **Find** (`Ctrl+F`) works in both modes: Monaco's native find in Source, a custom find-and-highlight in Reader.
- Window position, size, maximized state, and last theme persist across sessions.

3.5 Export

- **Export to HTML** — single self-contained file with CSS inlined and images optionally embedded as data URIs. Must render identically in a browser.
- **Export to PDF** — via `CoreWebView2.PrintToPdfAsync` with print-specific CSS (page margins, avoid breaking inside code blocks and table rows, headings kept with following content).
- **Print** (`Ctrl+P`).
- **Copy as rich text** — puts HTML on the clipboard in `CF_HTML` format so pasting into Word or Outlook keeps formatting. Small feature, disproportionately loved.

3.6 Command line

```
mdreader <file.md>                open in reader mode
mdreader <file.md> --source         open in source mode
mdreader <file.md> --export-html <out.html>
mdreader <file.md> --export-pdf <out.pdf>
mdreader --version
```

The export paths must work headlessly (no visible window) so they can be scripted in CI.

3.7 Settings

Persisted to %APPDATA%\mdreader\settings.json , with a real settings dialog:

default mode, theme, custom theme file, font family and size overrides, zoom, split-view default, line-ending policy, image loading policy (§5), extensions to register, "don't ask about default app again", telemetry (see below), and update-check preference.

Telemetry: none. Ever. No analytics, no crash reporting phone-home, no usage beacons. State this in the README and make it true in code. Optional, explicitly opt-in update check against the GitHub releases API, default off.

4. File association (get this exactly right)

This is the part naive implementations get wrong and then break users' machines.

4.1 What the installer registers

Register a ProgId and declare capability — **never claim the association outright.**

```

HKCU\Software\Classes\MdReader.Markdown.1
    (Default)                = "Markdown Document"
    FriendlyTypeName          = "Markdown Document"
    DefaultIcon               = "<INSTALLFOLDER>\mdreader.exe,1"
    shell\open\command        = "\"<INSTALLFOLDER>\mdreader.exe\" \"%1\""
    shell\edit\command        = "\"<INSTALLFOLDER>\mdreader.exe\" --source \"%1\""

HKCU\Software\Classes\.md\OpenWithProgids
    MdReader.Markdown.1      = (empty REG_SZ)           # ADDITIVE – never overwrite

HKCU\Software\Classes\Applications\mdreader.exe
    FriendlyAppName          = "mdreader"
    SupportedTypes\.md       = ""
    SupportedTypes\.markdown = ""

HKCU\Software\mdreader\Capabilities
    ApplicationName          = "mdreader"
    ApplicationDescription   = "Fast markdown reader and editor"
    FileAssociations\.md     = "MdReader.Markdown.1"
    FileAssociations\.markdown = "MdReader.Markdown.1"

HKCU\Software\RegisteredApplications
    mdreader                 = "Software\mdreader\Capabilities"

```

Extensions: .md and .markdown by default. .mdown , .mkd , .mkdn , .mdtxt , .mdtext are opt-in checkboxes in the installer and settings.

After any registration change, call `SHChangeNotify(SHCNE_ASSOCCHANGED, SHCNF_IDLIST, IntPtr.Zero, IntPtr.Zero)` .

4.2 What the installer must NOT do

Do not write to

`HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\FileExts\.md\UserChoice` . Since Windows 8 that key is protected by a per-user, per-extension hash. Writing it directly either fails, gets reverted, or gets the binary flagged as a hijacker by Defender and every AV vendor. Any StackOverflow answer suggesting otherwise is describing Windows 7. There is no supported way for an installer to set the default handler without the user's explicit action, and that is by design.

Do not overwrite `HKCU\Software\Classes\.md\{Default}` . That clobbers whatever handler the user already had. Use `OpenWithProgids` only.

4.3 How the user is asked

On first run (and only if `.md` is not already pointing at us and the user hasn't dismissed it permanently), show a **non-modal, dismissible bar** at the top of the window:

```
Make mdreader the default app for Markdown files? [ Set as default ] [ Not now ] [ Don't ask again ]
```

"Set as default" invokes `SHOpenWithDialog` with `OAIF_FORCE_OPEN_WITH | OAIF_EXEC` against the current document, which presents the standard Windows "How do you want to open this file?" dialog with the "Always use this app" checkbox — the user makes the choice, Windows records it correctly.

Fallback if that path fails or no document is open: launch `ms-settings:defaultapps?registeredAppUser=mdreader` and show a two-line instruction with a screenshot-free, text-only description of where to click.

Note that `IApplicationAssociationRegistrationUI::LaunchAdvancedAssociationUI` is deprecated on Windows 10+ and must not be used.

Also expose this from **Settings** → **File associations** so a user who dismissed it can come back, along with a live read-out of which app currently owns each extension.

4.4 Uninstall

Remove all keys written above. Do **not** touch `UserChoice`. Leave the user's `.md` handler in whatever state Windows falls back to.

5. Security posture

Assume a security-conscious reviewer. Markdown files arrive from repos, downloads, LLM output, and email attachments — treat every document as untrusted input.

1. **Sanitize before render.** Markdig may emit raw HTML passthrough. Run the generated HTML through `HtmlSanitizer` with an allowlist of tags and attributes. Strip `<script>`, event handlers (`on*`), `<iframe>`, `<object>`, `<embed>`, `<form>`, and `javascript:` / `vbscript:` / `data:` URIs (except `data:image/*` in the specific export path where we generate them ourselves). Raw-HTML-passthrough may be offered as an explicit per-document opt-in, defaulting off, and the sanitizer still runs even then.

2. **Content Security Policy** on the reader document: `default-src 'none'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' file: data:; font-src 'self'; connect-src 'none'` . Adjust minimally and document any relaxation with a comment explaining why.
3. **Remote resources blocked by default.** Remote images are a read-receipt/tracking vector. Block them, show a placeholder, and offer a per-document "Load remote images" action plus a global setting.
4. **Link handling.** All navigation inside the WebView2 is cancelled and routed to the host, which opens `http / https / mailto` in the default browser via `ShellExecute` . In-document anchors (`#heading`) scroll internally. Any other scheme — `file:` , `ms-` , custom protocols — is refused with a visible notice. Never let the WebView navigate away from the local document.
5. **WebView2 hardening.** `AreDevToolsEnabled = false` in Release, `AreDefaultContextMenusEnabled` customized, `IsGeneralAutofillEnabled = false` , `IsPasswordAutosaveEnabled = false` . Serve web assets via `SetVirtualHostNameToFolderMapping` with read-only access to the app's asset folder and the document's own directory only — not the whole filesystem.
6. **Path handling.** Canonicalize and validate document paths; refuse to resolve relative image paths that escape the document root more than N levels (configurable, default 3); handle UNC paths and long paths correctly.
7. **Mermaid** runs with `securityLevel: 'strict'` and `htmlLabels: false` .
8. **No elevation.** The app never requests admin. Per-user install by default.

Write a `SECURITY.md` documenting the threat model and a private disclosure address. Include an XSS-payload corpus in `fixtures/` and assert against it in tests — at minimum the standard markdown-XSS vectors: `[click](javascript:alert(1))` , `)` , raw `<img src=x onerror= ... >` , `<svg onload= ... >` , HTML-entity-encoded and case-mixed variants of each.

6. Performance targets

Measure these; do not estimate them. Add a benchmark project.

- Cold start to rendered first document: **< 1200ms** on a mid-range machine.
- Warm start (single-instance handoff, new tab): **< 300ms**.
- Re-render on save/reload for a 100KB document: **< 150ms**.

- A 5MB markdown file must open without freezing the UI — render progressively or virtualize, and show a "large document" notice with syntax highlighting deferred.
- Idle memory with one document open: under 200MB including the WebView2 process.

Optimizations to implement up front: warm the WebView2 environment during app startup on a background thread; cache the parsed Markdig pipeline (build it once, it is expensive); serve web assets from the virtual host rather than string-injecting the whole document; on re-render, swap only the document body rather than reloading the page.

7. Testing

- **Golden-file tests** (Verify): each fixture `.md` → sanitized HTML snapshot. Cover every enabled extension, plus pathological input: unclosed fences, deeply nested lists, 500-column tables, mixed line endings, BOM/no-BOM, UTF-8 vs UTF-16, emoji and CJK, a 5MB document.
- **Security tests**: every payload in the XSS corpus asserts sanitized output.
- **Shell tests**: registration writes exactly the expected keys and nothing else; uninstall removes exactly those keys; assert that `UserChoice` is never touched (this test is the guardrail — write it first).
- **Round-trip tests**: open → save with no edits → file bytes are identical (encoding, BOM, line endings, trailing newline).
- **Integration**: single-instance handoff, live reload debounce, dirty-state guard.
- **UI smoke tests** with FlaUI, minimal — launch, open a fixture, toggle modes, close.
- **Installer test** in CI: install the MSI silently on a clean Windows runner, assert registry state and that `mdreader.exe --version` works, uninstall, assert clean removal.

Target ≥80% line coverage on `MdReader.Core` and `MdReader.Shell`. UI coverage is not a goal.

8. Packaging and distribution

8.1 Installer — read before starting

Use **WiX Toolset v7**. Before writing the installer project, check the current WiX licensing terms and report back to me: WiX moved to a model involving an OpenSource Maintenance Fee with a revenue-based

threshold. I need to know whether this project, and separately my consultancy, falls above or below it. **Do not just start using it — surface the answer first.** If the terms are a problem, the approved alternatives are Inno Setup (free, mature, good file-association support) or MSIX (cleanest declarative associations, but signing makes GitHub distribution painful for an unsigned OSS project).

Installer requirements:

- Per-user install by default, no elevation, into `%LOCALAPPDATA%\Programs\mdreader`. Offer a per-machine option that does require elevation.
- Detect the WebView2 Evergreen Runtime; if absent, run the Evergreen Bootstrapper. Do not bundle the fixed-version runtime unless the bootstrapper path proves unworkable.
- Upgrade path: stable UpgradeCode, MajorUpgrade with proper scheduling, preserve settings.
- Optional Start Menu shortcut, optional desktop shortcut (default off), optional "Open with mdreader" context menu entry.
- Extension checkboxes for the opt-in file types.
- Silent install (`/qn`) with properties for every option — people will deploy this via Intune/SCCM.
- Clean uninstall including the settings folder only if the user opts in.

8.2 Signing

Unsigned installers trigger SmartScreen and get flagged. Wire the pipeline for signing but keep it optional so contributors can build without certificates. Document Azure Trusted Signing as the recommended low-cost option and leave the secrets as GitHub Actions inputs.

8.3 CI/CD (GitHub Actions)

- `ci.yml` — build, test, coverage, format check on every PR (windows-latest).
- `release.yml` — on tag: build Release, sign if secrets present, produce MSI + a portable ZIP, generate a changelog, create the GitHub Release, and attach SHA256SUMS.
- `winget.yml` — submit/update the winget manifest so `winget install mdreader` works. This is the single highest-leverage distribution step; do not skip it.
- Also produce a portable no-install ZIP build. Many corporate users cannot run MSIs.

8.4 Open-source hygiene

`README.md` with screenshots, a feature list, install instructions (winget, MSI, portable, build from source), and an explicit "what this is not" section. Plus: `LICENSE` (MIT), `CONTRIBUTING.md`, `SECURITY.md`,

CHANGELOG.md (Keep a Changelog), issue and PR templates, a THIRD-PARTY-NOTICES.md enumerating every bundled dependency and its license (Markdig, HtmlSanitizer, Monaco, highlight.js, Mermaid, KaTeX — check each license permits redistribution and note any attribution requirements), Semantic Versioning, and a .editorconfig .

9. Build phases — stop at each gate

Phase 1 — Core pipeline. MdReader.Core : Markdig configuration, sanitizer, HTML document assembly, theme CSS, source-line anchors. Full golden-file and XSS test suites. No UI. *Gate: dotnet test green, and show me the rendered HTML for the fixture set opened in a browser.*

Phase 2 — Reader shell. WPF window, WebView2 host, virtual host mapping, link interception, open file, drag-drop, zoom, theme following, live reload, recent files. *Gate: double-click-equivalent launch of a complex fixture, looking correct.*

Phase 3 — Source mode. Monaco integration, Ctrl+E toggle with scroll-position mapping, save with encoding/line-ending preservation, dirty state, find, split view. *Gate: edit → save → Reader reflects the change; file bytes round-trip cleanly.*

Phase 4 — Shell integration. MdReader.Shell registration, the first-run suggestion bar, SHOpenWithDialog path, settings page with live association read-out, single-instance + tabs. *Gate: registry state test passing, and the suggestion flow demonstrated end to end.*

Phase 5 — Export, CLI, settings dialog. HTML/PDF export, print, copy-as-rich-text, command line, full settings UI, custom theme loader.

Phase 6 — Packaging. Installer (after the §8.1 licensing answer), CI, release workflow, portable build, winget manifest, all documentation.

At every gate: full test suite, dotnet format --verify-no-changes , coverage report, a measurement against the §6 performance targets, and an explicit list of anything you stubbed, skipped, or were unsure about. **Do not paper over uncertainty — flag it.**

10. Ask me before deciding

- The WiX licensing answer (§8.1) — blocking, ask before Phase 6.
- Monaco vs CodeMirror 6 if the bundled asset size exceeds ~15MB.
- Whether to support `.mdx` (it is not really markdown and will render badly — my instinct is no).
- Anything where you would otherwise guess at a Windows shell API contract. Windows file association is full of deprecated advice; when unsure, say so rather than implementing something that looks plausible.

11. Non-goals

Not a note app. No wiki links, backlinks, graph view, tags, or vaults. No cloud sync, no accounts, no collaboration. No WYSIWYG editing — Source mode shows raw markdown, by design. No plugin system in v1 (custom CSS themes are the extensibility story). No telemetry. No AI features.